

PROYECTO

ESCUELA : TECNOLOGÍAS DE LA INFORMACIÓN
CARRERA : ARQUITECTURA DE DATOS EMPRESARIALES – BIG DATA
CURSO : 4364 LENGUAJE DE CIENCIA DE DATOS II
PERÍODO : 2026
CICLO : CUARTO

Instituto de Educación
Superior privado
CIBERTEC

Integrantes:

- | | |
|--|------------|
| 1. Giomar Alfredo Guerra Lazaro | i202400989 |
| 2. Elizabeth Jenny Cordova Carhuaricra | i202401903 |
| 3. Miguel Angel Castillo Palomino | i202500007 |
| 4. Thania Colan Penadillo | i202502109 |
| 5. Yenny Malyeris Salas Torres | i202415629 |

Docente:

- CARLOS ALBERTO ACOSTA CORTEZ

Tema:

- Dashboard de criptomonedas (API)

2026

Contenido

1. RESUMEN	3
2. INTRODUCCIÓN	3
3. JUSTIFICACIÓN DEL PROYECTO	4
4. OBJETIVOS DEL PROYECTO	4
5. DEFINICIÓN Y ALCANCE DEL PROYECTO	5
6. PLAN DE TRABAJO (HITOS)	6
7. ARQUITECTURA	6
10. RECOMENDACIONES	7
11. GLOSARIO	8

1. RESUMEN

El presente proyecto consiste en el diseño e implementación de una Data App interactiva que muestra el comportamiento de criptomonedas en tiempo real, consumiendo datos de la API pública de CoinGecko. La solución está construida íntegramente en Python, utilizando Streamlit como framework de visualización web y Pandas para la transformación y limpieza de datos.

La aplicación resuelve un problema concreto: las personas no cuentan con una forma clara, accesible y gratuita para analizar el comportamiento del mercado de criptomonedas en tiempo real. A través de un pipeline ETL que extrae datos del endpoint /coins/markets de CoinGecko, transforma las variables relevantes y las presenta en un dashboard interactivo, la plataforma permite visualizar precios actuales, tendencias de capitalización de mercado y variación porcentual en las últimas 24 horas de las 10 principales criptomonedas.

La aplicación está desplegada públicamente en Streamlit Community Cloud, accesible desde cualquier dispositivo sin necesidad de instalación local, y su código fuente está disponible en el repositorio de GitHub del equipo.

2. INTRODUCCIÓN

El mercado de activos digitales ha crecido exponencialmente en la última década, consolidándose como una clase de activos con capitalización global que ha superado los dos billones de dólares en períodos de alta actividad. A diferencia de los mercados financieros tradicionales, el ecosistema de criptomonedas opera de forma continua las 24 horas del día, los 7 días de la semana, generando un flujo constante de datos que resulta difícil de monitorear sin herramientas tecnológicas adecuadas.

En este contexto, la capacidad de consumir, procesar y visualizar datos provenientes de APIs financieras en tiempo real se convierte en una competencia fundamental para el profesional de ciencia de datos. Plataformas como CoinGecko ofrecen APIs REST de acceso público y gratuito que proveen información estructurada en formato JSON sobre miles de activos digitales.

Este proyecto integra de forma práctica las competencias desarrolladas a lo largo del curso: consumo de APIs con la librería requests, manipulación de datos con Pandas y construcción de interfaces web interactivas con Streamlit. El resultado es una aplicación funcional,

desplegada en la nube, que centraliza el análisis del mercado criptográfico en un entorno visual intuitivo y accesible para cualquier usuario.

3. JUSTIFICACIÓN DEL PROYECTO

A. Justificación del Problema

Las personas interesadas en el mercado de criptomonedas no disponen de una herramienta propia, personalizable y gratuita que les permita analizar el comportamiento de los activos en tiempo real. Las plataformas comerciales existentes (CoinMarketCap Pro, Bloomberg Terminal) tienen costos elevados o presentan interfaces sobrecargadas que dificultan el análisis rápido y directo.

B. Justificación Técnica

El proyecto permite aplicar de forma integrada el flujo completo de ciencia de datos sobre datos financieros reales: extracción mediante una API REST pública, transformación y limpieza con Pandas (Data Wrangling) y visualización interactiva con Streamlit. Este pipeline ETL end-to-end consolida las competencias clave del curso en un caso de uso con impacto real y verificable.

C. Justificación Educativa y de Replicabilidad

El proyecto está diseñado para ser completamente reproducible: el entorno de trabajo se gestiona con Conda para garantizar el aislamiento de dependencias, el código está versionado en GitHub y la aplicación se despliega de forma gratuita en Streamlit Community Cloud. Cualquier estudiante puede clonar el repositorio, configurar el entorno siguiendo los pasos documentados y ejecutar la aplicación en minutos.

4. OBJETIVOS DEL PROYECTO

Desarrollar una Data App en Python con Streamlit que consuma datos en tiempo real desde la API de CoinGecko, permitiendo visualizar precios, tendencias y capitalización de mercado de las principales criptomonedas de forma interactiva y accesible.

4.2 Objetivos Específicos

- Implementar un módulo de ingesta (ingest.py) que consuma el endpoint /coins/markets de CoinGecko mediante la librería requests y persista los datos crudos en formato CSV.
- Desarrollar un módulo de transformación (transform.py) que aplique técnicas de Data Wrangling: selección de variables relevantes, renombrado de columnas y eliminación de valores nulos.
- Construir la interfaz de la Data App (app.py) con Streamlit, mostrando la tabla de datos, un gráfico de barras de precios y métricas KPI clave (precio promedio).
- Incorporar mejoras de usabilidad: filtro por criptomoneda mediante selectbox y visualización del Top 10 por capitalización de mercado.
- Desplegar la aplicación en Streamlit Community Cloud a partir del repositorio de GitHub del equipo, garantizando acceso público y continuo a la herramienta.

5. DEFINICIÓN Y ALCANCE DEL PROYECTO

El proyecto consiste en el diseño, desarrollo y despliegue de una Data App de monitoreo del mercado de criptomonedas. La solución opera bajo el siguiente flujo de datos:

- Extracción: Consumo del endpoint /coins/markets de la API de CoinGecko (parámetros: vs_currency=usd, order=market_cap_desc, per_page=50).
- Transformación: Selección de las columnas name, current_price, market_cap y price_change_percentage_24h. Renombrado a nombres en español (crypto, precio, capitalización, cambio_24h). Eliminación de filas con valores nulos.
- Carga: Persistencia del dataset limpio en data/crypto.csv para consumo por la capa de visualización.
- Visualización: Dashboard en Streamlit con tabla de datos, gráfico de barras de precios, métrica KPI de precio promedio y filtro interactivo por criptomoneda.

- Despliegue: Publicación de la aplicación en Streamlit Community Cloud mediante integración continua con GitHub.

ALCANCE DEL PROYECTO

- Monitoreo y visualización del Top 10 de criptomonedas por capitalización de mercado.
- Visualización de precio actual (USD), capitalización de mercado y variación porcentual 24h.
- Filtro interactivo por nombre de criptomoneda.
- KPI de precio promedio del mercado.
- Despliegue público y gratuito en Streamlit Community Cloud.

6. PLAN DE TRABAJO (HITOS)

Flujo: Origen → ETL → Almacenamiento → Visualización → Seguridad/Deploy

1. Ingesta (Data Source)

- API (qué fuente usan)
- cómo se obtiene la data ([ingesta.py](#))
- problema inicial (datos crudos)

“Extraemos datos desde una API externa en tiempo real”

2. Transformación (ETL)

- limpieza de datos
- selección de columnas
- KPIs base
- [transform.py](#)

“Convertimos datos crudos en información útil”

3. Almacenamiento (CSV → Parquet)

- por qué CSV no es suficiente

- cambio a Apache Parquet
- uso de **pyarrow**
- eficiencia

“Optimizamos rendimiento usando Parquet, un formato columnar”

4. Dashboard (Streamlit)

- KPIs
- gráficos
- filtros
- diseño tipo BI

“Visualizamos datos para toma de decisiones”

5. Deploy + Seguridad

- Git + GitHub
- deploy en Streamlit Cloud
- login con **secrets**

“Hacer el sistema accesible y seguro en la nube”

7. ARQUITECTURA

La solución sigue una arquitectura de tres capas: extracción de datos desde la API, transformación con Python/Pandas y presentación con Streamlit. El siguiente diagrama resume las tecnologías de cada capa:

Capa	Tecnología	Función
Fuente de datos	CoinGecko API v3 (REST, JSON)	Provee precios, market cap y variación 24h de 50+ criptos
Extracción	Python + requests	Consumo del endpoint /coins/markets con parámetros configurables
Transformación	Python + Pandas	Limpieza de columnas, renombrado, eliminación de nulos
Almacenamiento	CSV local (data/crypto.csv)	Persistencia del dataset procesado para carga en la app

Visualización	Streamlit + gráficos nativos	Dashboard interactivo con tabla, gráfico de barras, KPIs y filtros
Despliegue	Streamlit Community Cloud + GitHub	Publicación pública del app sin costo de infraestructura
Entorno local	Conda (crypto_env) + VS Code	Aislamiento de dependencias y desarrollo reproducible

7.1 Estructura del Proyecto

El repositorio sigue una estructura modular que separa responsabilidades por carpeta:

```

proyecto_crypto/
├── data/
│   └── crypto.csv
├── src/
│   ├── ingest.py
│   └── transform.py
├── app/
│   └── app.py
├── requirements.txt
└── README.md

```

8. PRODUCTOS Y ENTREGABLES

A continuación se presenta el código de cada módulo del proyecto, organizado por etapa del pipeline de datos.

8.1 Módulo de Ingesta — src/ingest.py

Consume el endpoint /coins/markets de CoinGecko y guarda el resultado crudo en formato CSV. La función `get_data()` es reutilizable y puede llamarse desde cualquier otro módulo.

```

# ingest.py
import requests
import pandas as pd

url = "https://api.coingecko.com/api/v3/coins/markets"
params = {"vs_currency": "usd"}

data = requests.get(url, params=params).json()
df = pd.DataFrame(data)
df.to_csv("data/raw_crypto.csv", index=False)

```



```
print(df.head())
```

8.2 Módulo de Transformación — src/transform.py

Aplica las técnicas de Data Wrangling: selección de variables relevantes, renombrado de columnas a español y eliminación de filas con valores nulos.

```
import pandas as pd

# leer datos crudos
df = pd.read_csv("data/raw_crypto.csv")

# seleccionar columnas importantes
df = df[['name', 'current_price', 'market_cap', 'price_change_percentage_24h']]

# renombrar
df = df.rename(columns={
    'name': 'crypto',
    'current_price': 'precio',
    'market_cap': 'capitalizacion',
    'price_change_percentage_24h': 'cambio_24h'
})

# limpiar
df = df.dropna()

# Después de limpiar los datos
df['cambio_24h'] = df['cambio_24h'].round(2)
df['cambio_24h'] = df['cambio_24h'].replace(-0.00, 0.00)

# guardar limpio
df.to_csv("data/crypto.csv", index=False)      # respaldo
df.to_parquet("data/crypto.parquet", index=False) # producción

print(df.head())
```

8.3 Data App — app/app.py

Interfaz principal del dashboard. Carga el dataset transformado y muestra la tabla de datos, el gráfico de barras de precios, los KPIs y el filtro interactivo.

```
import streamlit as st
import pandas as pd
import subprocess
from auth import login

auth = login()
```

```

if not auth:
    st.warning("Acceso restringido")
    st.stop()

subprocess.run(["python", "src/ingesta.py"])
subprocess.run(["python", "src/transform.py"])

# configuración
st.set_page_config(layout="wide")

# cargar datos
df = pd.read_parquet("data/crypto.parquet")

# =====
# 1. TÍTULO
# =====
st.title("Dashboard del Mercado Cripto")
st.caption("Datos en tiempo real desde API - Análisis de capitalización y variación")

st.divider()

# =====
# 2. KPIs
# =====
top = df.sort_values(by='capitalizacion', ascending=False).iloc[0]
top_gain = df.sort_values(by='cambio_24h', ascending=False).iloc[0]
top_loss = df.sort_values(by='cambio_24h').iloc[0]

col1, col2, col3, col4 = st.columns(4)

col1.metric("Market Cap Total", f"${df['capitalizacion'].sum():.0f}")
col2.metric("Cambio Promedio 24h", f"{df['cambio_24h'].mean():.2f}%")
col3.metric("Top Cripto", top['crypto'])
col4.metric("Precio Top", f"${top['precio']:.2f}")

st.divider()

# =====
# 3. GRÁFICOS
# =====

col1, col2 = st.columns(2)

top10 = df.sort_values(by='capitalizacion', ascending=False).head(10)

with col1:
    st.subheader("Top 10 por Capitalización")
    st.bar_chart(top10.set_index('crypto')['capitalizacion'])

with col2:

```

```

st.subheader("Variación 24h (%)")
st.bar_chart(top10.set_index('crypto')['cambio_24h'])

st.divider()

# =====
# 4. FILTRO
# =====
st.subheader("🔍 Análisis por criptomoneda")

crypto = st.selectbox("Selecciona criptomoneda", df['crypto'])

row = df[df['crypto'] == crypto].iloc[0]

col1, col2, col3 = st.columns(3)

col1.metric("Precio", f"${row['precio']:.2f}")
col2.metric("Market Cap", f"${row['capitalizacion']:.0f}")
col3.metric("Cambio 24h", f"{row['cambio_24h']:.2f}%")

st.divider()

# =====
# 5. TABLA DETALLADA
# =====
def color_cambio(val):
    if val > 0:
        return 'color: green'
    elif val < 0:
        return 'color: red'
    return 'color: gray'

styled_df = df.style.format({
    "precio": "${:.2f}",
    "capitalizacion": "${:.0f}"
})

styled_df = styled_df.map(color_cambio, subset=["cambio_24h"])
st.dataframe(styled_df)

```

8.4 Instrucciones de Ejecución Local

Para reproducir el proyecto de forma local se deben seguir los siguientes pasos en orden:

- **Clonar el repositorio desde GitHub:**

D: git clone https://github.com/EnsaladaRusa/crypto-dashboard.git cd crypto-dashboard

- **Crear y activar el entorno Conda:**

```
conda create -n crypto_env python=3.10 -y
conda activate crypto_env
```

- **Instalar librerías :**

```
pip install -r requirements.txt
```

- **Crear carpeta de /secrets :**

```
mkdir .streamlit
```

- **Crear archivo de credenciales**

```
notepad .streamlit\secrets.toml
```

Pegas dentro del archivo secrets.toml:

```
USER = "streamdash"
PASS = "streamdash123"
```

Ejecutar la app

```
streamlit run app/app.py
```

9. CONCLUSIONES

- **Conclusión sobre el problema resuelto:** El proyecto valida que es posible construir una solución funcional al problema de análisis de criptomonedas en tiempo real utilizando únicamente herramientas de código abierto y APIs públicas gratuitas. La Data App centraliza en un solo punto visual la información que el usuario normalmente debería buscar en múltiples plataformas.
- **Conclusión técnica sobre el pipeline ETL:** La separación en módulos independientes (ingest.py, transform.py, app.py) demuestra que una arquitectura modular facilita el mantenimiento, la depuración y la escalabilidad del proyecto. El proceso de Data Wrangling con Pandas resultó eficiente para manejar las inconsistencias del JSON retornado por la API (columnas irrelevantes, valores nulos y tipos de datos mixtos).
- **Conclusión sobre el despliegue y la replicabilidad:** El uso de Conda para gestión del entorno y GitHub como repositorio central garantiza que cualquier integrante del equipo pueda reproducir el proyecto en su máquina local en menos de 10 minutos. El despliegue en Streamlit Community Cloud elimina la necesidad de infraestructura propia, reduciendo la barrera de acceso a herramientas de análisis de datos de calidad profesional.

10.RECOMENDACIONES

- **Agregar actualización automática de datos:** Implementar `st.cache_data` con TTL de 60 segundos y un botón de 'Actualizar datos' para que el dashboard consuma datos frescos de la API sin necesidad de reiniciar la aplicación, eliminando la dependencia del archivo CSV estático.
- **Incorporar análisis histórico con gráficos de velas (OHLCV):** Como mejora de siguiente fase, integrar el endpoint `/coins/{id}/market_chart` de CoinGecko para obtener datos históricos de precios y construir gráficos de velas japonesas con Plotly, enriqueciendo significativamente el análisis disponible.
- **Implementar análisis de correlación entre activos:** Agregar un módulo que calcule el coeficiente de correlación de Pearson entre los retornos diarios de las criptomonedas seleccionadas y lo visualice como un mapa de calor (heatmap), permitiendo identificar relaciones entre activos para estrategias de diversificación.
- **Migrar el almacenamiento de CSV a base de datos:** Reemplazar el archivo `crypto.csv` por una base de datos SQLite o PostgreSQL para habilitar consultas históricas, comparativas temporales y un mayor volumen de datos sin degradar el rendimiento de la aplicación.
- **Agregar alertas de variación de precio:** Incorporar un módulo de notificaciones que alerte al usuario cuando una criptomoneda supere un umbral de variación configurable (por ejemplo, más del 5% en 24h), utilizando la API de Telegram Bot para el envío de mensajes automáticos.

11. GLOSARIO

Término	Definición
API (Application Programming Interface)	Interfaz de programación que permite la comunicación entre sistemas de software mediante peticiones y respuestas en formato estándar (JSON/XML) a través del protocolo HTTP.
CoinGecko API	Servicio REST público y gratuito que provee datos en tiempo real de más de 10,000 criptomonedas: precios, capitalización de mercado, volumen de negociación y variación porcentual.
Criptomonedas	Activo digital descentralizado basado en tecnología blockchain que utiliza criptografía para asegurar transacciones y controlar la emisión de nuevas unidades.
Data App	Aplicación web orientada a datos que combina procesamiento, análisis y visualización en una interfaz interactiva, sin requerir conocimientos de desarrollo web tradicional.
Data Wrangling	Proceso de limpieza, transformación y estructuración de datos crudos en un formato adecuado para su análisis y visualización.
Streamlit	Framework de Python de código abierto para construir aplicaciones web de datos de manera rápida, usando únicamente código Python sin necesidad de HTML o JavaScript.
Pandas	Librería de Python para manipulación y análisis de datos estructurados. Permite cargar, transformar, filtrar y agregar datos mediante estructuras tipo DataFrame.
requests	Librería de Python para realizar peticiones HTTP de forma sencilla. Utilizada para consumir la API de CoinGecko y obtener los datos en formato JSON.
Market Cap (Capitalización de mercado)	Valor total de una criptomoneda en el mercado, calculado como: precio actual $\times$ total de monedas en circulación.
Variación 24h	Cambio porcentual en el precio de una criptomoneda durante las últimas 24 horas. Valor positivo indica alza; negativo indica caída.
CSV (Comma-Separated Values)	Formato de archivo de texto plano donde los datos se separan por comas. Utilizado para almacenar el dataset transformado de criptomonedas.
Entorno virtual (conda)	Espacio de trabajo aislado que contiene su propia instalación de Python y librerías, evitando conflictos de dependencias entre proyectos.
KPI (Key Performance Indicator)	Indicador clave de rendimiento. En el dashboard: precio promedio, mayor capitalización y mayor variación de precio en 24 horas.
ETL (Extract, Transform, Load)	Proceso de extracción de datos desde la API, transformación y limpieza con Pandas, y carga al entorno de visualización de Streamlit.
GitHub	Plataforma de alojamiento de repositorios de código con control de versiones Git. Sirve como fuente de despliegue continuo para Streamlit Community Cloud.

